

Test Plan and Review Report

Scope and method

The review inspected source files, route/controller/model wiring, package manifests/lockfiles, and the latest public commit. It executed static/build checks that do not require a running MongoDB instance. No source code was modified.

Executed checks (18 July 2026)

Check	Command/method	Result
Frontend clean dependency install	<code>npm ci</code>	Passed
Frontend production build	<code>npm run build</code>	Passed; 138 modules transformed
Build output	Vite size report	JS 395.34 kB (114.16 kB gzip); CSS 15.32 kB (3.90 kB gzip)
Frontend lint	<code>npm run lint</code>	Completed with 30 warnings, 0 reported errors
Backend syntax	<code>node --check</code> on project <code>.js</code> files	Passed
Client production dependency audit	<code>npm audit --omit=dev</code>	0 known vulnerabilities reported
Server production dependency audit	<code>npm audit --omit=dev</code>	0 known vulnerabilities reported
Unit tests	Repository inspection	No test suite/script present
API integration tests	Repository inspection	No test suite/script present
E2E tests	Repository inspection	No test suite/config present
Live database behavior	Not run	MongoDB/test environment not included

Dependency audit results are time-specific and do not establish that application code is secure.

Lint warning summary

Warnings include unused imports/variables/parameters, missing React hook dependencies, and context/component co-location warnings affecting Fast Refresh. These are not build blockers, but missing hook dependencies can cause stale behavior and should be reviewed rather than mechanically suppressed.

Required test pyramid

Backend unit tests

- JWT generation/verification failure modes.
- Password hashing and matching.
- Role middleware for every role/route combination.
- Ticket/transaction ID collision retry behavior after it is implemented.
- Booking state transition matrix.
- Age, gender, amount, seat, price, date, and time validation.
- Analytics calculations including empty and orphaned records.

API integration tests

- Registration always creates USER for public calls.
- Duplicate email returns a stable conflict/validation response.
- Authenticated profile reads/updates only the caller.
- Temple search escaping and filter behavior.
- Organizer access is limited to assigned temple IDs.
- Review update does not duplicate a caller's review.
- Slot creation rejects negative capacity/price and invalid dates/times.
- Booking hold is atomic and rolls back if booking creation fails.
- Confirmation and cancellation are owner/role-authorized and idempotent.
- Expiration restores capacity exactly once.
- Deletes follow the chosen retention/cascade policy.

Frontend tests

- Authentication restoration/logout on 401.
- Route guards and role redirects.
- Booking devotee count, age/gender validation, timer, expiry, and ticket rendering.
- Donation amount/purpose and simulated-payment labeling.
- Dashboard loading, empty, error, and unauthorized states.
- Keyboard focus trap/return for modals and screen-reader labels.

E2E scenarios

ID	Scenario	Expected outcome
E2E-01	Guest searches by name/location	Matching temples appear
E2E-02	Devotee registers/logs in	USER session created; privileged roles unavailable
E2E-03	Devotee holds and confirms two seats	Capacity decreases by two; booked ticket appears
E2E-04	Hold expires	Status becomes Expired; seats return exactly once
E2E-05	Pending hold cancelled	Seats return immediately; status becomes Cancelled

ID	Scenario	Expected outcome
E2E-06	Two users compete for final seat	Exactly one succeeds; capacity never negative
E2E-07	User tries admin API	403
E2E-08	Organizer queries another temple	403 or empty by assignment policy
E2E-09	Donation simulation completes	Receipt/history record uses server-generated transaction ID
E2E-10	Admin manages event/maintenance	Authorized updates persist and audit record is created

Concurrency test

Create a slot with one available seat, authenticate 20 users, synchronize 20 booking requests, and assert one **Pending** booking, nineteen capacity failures, `availableSeats === 0`, and no orphaned holds. Repeat through confirm, expire, and cancel transitions. The current read-modify-save implementation is expected to be vulnerable to race conditions, so this should be a release gate after atomic allocation is implemented.

Exit criteria

- No P0/P1 security or data-consistency defects open.
- All automated suites pass in CI.
- Booking concurrency test passes repeatedly.
- Coverage thresholds are agreed and met for critical business logic.
- Accessibility and supported-browser matrix pass.
- Staging smoke, migration, backup/restore, deployment, and rollback tests pass.